

Guía de Práctica de Laboratorio: Sistema de Auditoría y Control de Sesiones de Red

Asignatura: Programación Avanzada y Algoritmos | **Tema 2.4:** Ámbitos de Atributos, Métodos, Objetos y Clases | **Duración:** 80 min

1. Objetivos Académicos de la Sesión

En esta práctica el estudiante consolidará la gestión de tiempos de vida en memoria, el alcance de los identificadores y las estructuras avanzadas de C++ a través del desarrollo de un sistema de auditoría de red:

- **Comprender y aplicar el ámbito de clase (static):** Implementar atributos y métodos estáticos compartidos dinámicamente por todas las instancias.
- **Resolver el ocultamiento de variables (shadowing):** Utilizar el puntero explícito `this->` para desambiguar miembros de instancia homónimos con parámetros locales.
- **Estructurar código en Espacios de Nombres (namespace):** Encapsular tipos y clases dentro de un ámbito lógico corporativo (`SeguridadNet::ControlAcceso`).
- **Analizar el ciclo de vida y la pila de ejecución:** Monitorear la creación y destrucción automática de objetos al entrar y salir de bloques de código `{}`.

2. Escenario Técnico y Planteamiento del Problema

Una firma de ciberseguridad requiere un módulo auditor en C++ dentro del espacio de nombres `SeguridadNet::ControlAcceso` para gestionar el acceso concurrente a servidores críticos. El sistema debe garantizar que nunca existan más de un número máximo permitido de sesiones simultáneas (`MAX_SESIONES = 3`).

Cada sesión de red representa una instancia de la clase `SesionRed` con un identificador único correlativo, nombre de usuario e IP de origen. Dado que el límite de sesiones es global para todo el sistema, este valor no puede almacenarse en atributos de instancia individuales; debe gestionarse mediante celdas de memoria de ámbito de clase (`static`). Asimismo, los parámetros del constructor comparten exactamente el mismo nombre que los miembros privados, requiriendo una desambiguación sintáctica mediante el puntero `this->`.

Resumen de Especificación de Ámbitos para la Práctica:

Identificador / Elemento	Nivel de Ámbito	Ubicación en Memoria	Propósito y Comportamiento
<code>SeguridadNet::ControlAcceso</code>	Namespace	Tabla de Símbolos	Previene colisiones de nombres corporativos.
<code>sesionesActivas</code>	Clase (<code>static</code>)	Segmento de Datos	Contador global compartido por todas las instancias.
<code>MAX_SESIONES</code>	Clase (<code>static const</code>)	Segmento de Datos	Constante global que limita las sesiones simultáneas.
<code>usuario</code> , <code>ipOrigen</code>	Instancia (<code>private</code>)	Stack / Heap	Atributos individuales de cada objeto <code>SesionRed</code> .
<code>usuario</code> (parámetro)	Local / Bloque	Stack de Función	Oculto al atributo; desambiguado con <code>this-></code> .

3. Código Fuente C++ Completo y Comentado

El siguiente programa contiene la implementación modular completa del ejercicio. Transcriba el código por bloques y observe la interacción de los ámbitos:

```

// PARTE 1: Librerías y Definición de la Clase dentro del Namespace
#include <iostream>
#include <string>
#include <vector>
#include <stdexcept>

namespace SeguridadNet {
namespace ControlAcceso {

    class SesionRed {
    private:
        // --- ATRIBUTOS DE INSTANCIA (Ámbito de Miembro) ---
        int idSesion;
        std::string usuario;
        std::string ipOrigen;

        // --- ATRIBUTOS DE CLASE (Ámbito de Clase / Static) ---
        static int sesionesActivas;
        static int contadorCorrelativoId;
        static const int MAX_SESIONES = 3; // Límite estricto de concurrencia

    public:
        // Constructor parametrizado con SHADOWING (Ocultamiento)
        SesionRed(std::string usuario, std::string ipOrigen) {
            if (sesionesActivas >= MAX_SESIONES) {
                throw std::runtime_error("ERROR 403: Límite de sesiones alcanzado ("
                    + std::to_string(MAX_SESIONES) + ").");
            }

            // Desambiguación explícita mediante el puntero this->
            this->usuario = usuario; // Atributo = Parámetro local
            this->ipOrigen = ipOrigen; // Atributo = Parámetro local

            contadorCorrelativoId++;
            this->idSesion = contadorCorrelativoId;
            sesionesActivas++;

            std::cout << " [+] [SESIÓN INICIADA] ID: " << idSesion
                << " | Usuario: " << this->usuario
                << " | IP: " << this->ipOrigen
                << " (Activas: " << sesionesActivas << "/" << MAX_SESIONES << ")\n";
        }

        // Destructor: Se ejecuta al salir del Ámbito Local del objeto
        ~SesionRed() {
            sesionesActivas--;
            std::cout << " [-] [SESIÓN CERRADA] ID: " << idSesion
                << " | Usuario: " << usuario
                << " (Activas restantes: " << sesionesActivas << ")\n";
        }

        // Métodos Estáticos (Ámbito de Clase)
        static int obtenerSesionesActivas() { return sesionesActivas; }
        static int obtenerMaxSesiones() { return MAX_SESIONES; }

        void mostrarInfo() const {
            std::cout << "      Detail -> ID: " << idSesion << " | User: " << usuario
                << " | IP: " << ipOrigen << "\n";
        }
    };
};

```

```
// PARTE 2: Inicialización de Miembros Estáticos fuera de la clase
// Asignación de memoria única en el Data Segment
int SesionRed::sesionesActivas = 0;
int SesionRed::contadorCorrelativoId = 1000;

} // namespace ControlAcceso
} // namespace SeguridadNet

// PARTE 3: Función Main y Pruebas de Ámbito
int main() {
    using namespace SeguridadNet::ControlAcceso;

    std::cout << "===== " << std::endl;
    std::cout << " AUDITORÍA DE SESIONES - DEMOSTRACIÓN DE ÁMBITOS Y STATIC " << std::endl;
    std::cout << "===== " << std::endl;

    std::cout << "\n1. Estado inicial de clase (sin objetos):" << std::endl;
    std::cout << "    Sesiones activas: " << SesionRed::obtenerSesionesActivas()
        << " / " << SesionRed::obtenerMaxSesiones() << std::endl;

    try {
        std::cout << "\n2. Creación de sesiones en ámbito del main:" << std::endl;
        SesionRed s1("admin_mendoza", "192.168.1.10");
        SesionRed s2("analista_garcia", "192.168.1.15");

        std::cout << "\n3. Demostración de ÁMBITO DE BLOQUE LOCAL { ... }:" << std::endl;
        { // Inicio de bloque local
            std::cout << "    ---> Entrando al bloque interno..." << std::endl;
            SesionRed s3("invitado_temporal", "10.0.0.5");
            std::cout << "    ---> Activas en bloque: " << SesionRed::obtenerSesionesActivas() << "\n";
            std::cout << "    ---> Saliendo del bloque interno..." << std::endl;
        } // s3 destruido automáticamente aquí por salir de ámbito local

        std::cout << "    ---> Fuera del bloque. Activas restantes: "
            << SesionRed::obtenerSesionesActivas() << std::endl;

        std::cout << "\n4. Intento de exceder el límite de sesiones (MAX=3):" << std::endl;
        SesionRed s4("operador_rodriguez", "192.168.1.20"); // Activa #3
        SesionRed s5("intruso_excedente", "10.0.0.99");      // Lanza excepción

    } catch (const std::exception& e) {
        std::cout << "\n [EXCEPCIÓN CAPTURADA EN MAIN] " << e.what() << std::endl;
    }

    std::cout << "\n5. Verificación final tras cierre de bloque try:" << std::endl;
    std::cout << "    Sesiones activas finales: " << SesionRed::obtenerSesionesActivas() << std::endl;

    return 0;
}
```

4. Guía de Ejecución Paso a Paso (80 Minutos)

Fase 1: Estructuración y Espacios de Nombres (00' - 20')

1. Crear el proyecto en C++ y definir la jerarquía de espacios de nombres namespace SeguridadNet::ControlAcceso.
2. Declarar la clase SesionRed identificando la diferencia conceptual entre los atributos privados de objeto (usuario, ipOrigen) y las variables de clase (static int sesionesActivas).

Fase 2: Desambiguación con this-> e Inicialización Estática (20' - 40')

1. Implementar el constructor utilizando intencionalmente nombres de parámetros idénticos a los miembros (usuario, ipOrigen).
2. Aplicar la desambiguación sintáctica mediante `this->usuario = usuario;`.
3. Definir la inicialización externa de las variables estáticas fuera de la clase: `int SesionRed::sesionesActivas = 0;`.

Fase 3: Análisis de Ámbito de Bloque y Excepciones (40' - 60')

1. En la función `main()`, construir un bloque condicional explícito `{ ... }` para observar cómo la variable `s3` llama automáticamente a su destructor al salir de la llave de cierre.
2. Verificar en la consola el comportamiento del contador `sesionesActivas` antes, dentro y después del bloque.
3. Comprobar que al intentar crear la cuarta sesión simultánea, el sistema bloquea el registro mediante la excepción `std::runtime_error`.

Fase 4: Evaluación y Verificación de la Trampa Pedagógica (60' - 80')

1. Responder el cuestionario de control y verificar los errores de enlazado al violar las reglas de inicialización estática.

¡ATENCIÓN! LA TRAMPA PEDAGÓGICA DE LA INICIALIZACIÓN ESTÁTICA

Si un estudiante intenta inicializar la variable estática dentro del cuerpo de la clase en el archivo de cabecera (.h), por ejemplo: `static int sesionesActivas = 0;`, el compilador o enlazador (linker) emitirá un error crítico. **Razón técnica:** En C++, la declaración dentro de la clase es solo un anuncio de existencia. La reserva real de espacio de memoria en el *Data Segment* debe realizarse explícitamente fuera de la definición de la clase mediante la sintaxis: `int SeguridadNet::ControlAcceso::SesionRed::sesionesActivas = 0;`.

5. Cuestionario de Control y Evaluación Teórica

Pregunta 1: ¿Por qué la variable `sesionesActivas` no puede inicializarse dentro del constructor de la clase `SesionRed`?

Respuesta esperada: Porque el constructor se ejecuta cada vez que se crea una nueva instancia. Si se asignara un valor fijo en el constructor (ej. `sesionesActivas = 0`), se sobrescribiría el conteo acumulado global por las demás instancias activas.

Pregunta 2: ¿Qué sucede exactamente con el objeto `s3` cuando la ejecución del programa cruza la llave de cierre } del bloque interno?

Respuesta esperada: Al finalizar el ámbito local del bloque, la memoria de la Pila (Stack) asignada a `s3` se libera automáticamente, invocando de forma implícita a su destructor `~SesionRed()`, el cual decrementa el contador estático.

Pregunta 3: ¿Por qué es obligatoria la sintaxis `this->usuario = usuario;` dentro del constructor?

Respuesta esperada: Debido al ocultamiento de variables (*shadowing*). El parámetro local `usuario` tiene precedencia sobre el atributo miembro. El puntero `this->` le indica al compilador que se está modificando la celda de memoria de la instancia.

6. Rúbrica de Evaluación Académica (Ibero Puebla)

Criterio de Evaluación	Sobresaliente (10)	Satisfactorio (8)	Insuficiente (5)
Miembros Estáticos (35%)	Implementa atributos y métodos static con inicialización externa correcta y control estricto de concurrencia.	Declara miembros estáticos pero comete errores sintácticos de inicialización inicial fuera de la clase.	Confunde atributos de instancia con variables de clase o utiliza variables globales desprotegidas.
Desambiguación y Ámbitos (35%)	Aplica el puntero this-> para resolver <i>shadowing</i> y demuestra el ciclo de vida en bloques de código {}.	Resuelve el shadowing renombrando parámetros en lugar de utilizar el puntero de instancia this->.	No logra comprender el ámbito de bloque o el tiempo de vida automático de los objetos en la Pila.
Espacios de Nombres y Errores (30%)	Encapsula la solución en el namespace asignado y gestiona el límite de sesiones mediante excepciones.	Utiliza espacios de nombres pero omite el manejo de excepciones para el límite de sesiones.	No implementa espacios de nombres o presenta errores de compilación no resueltos.